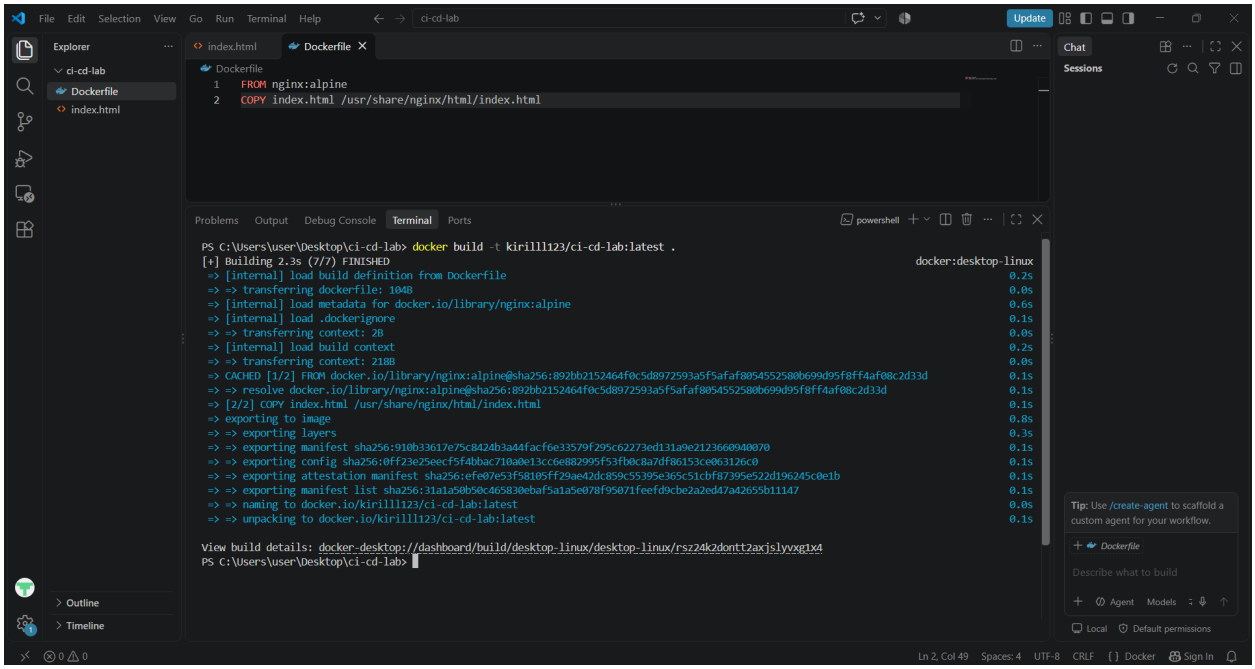
1.1



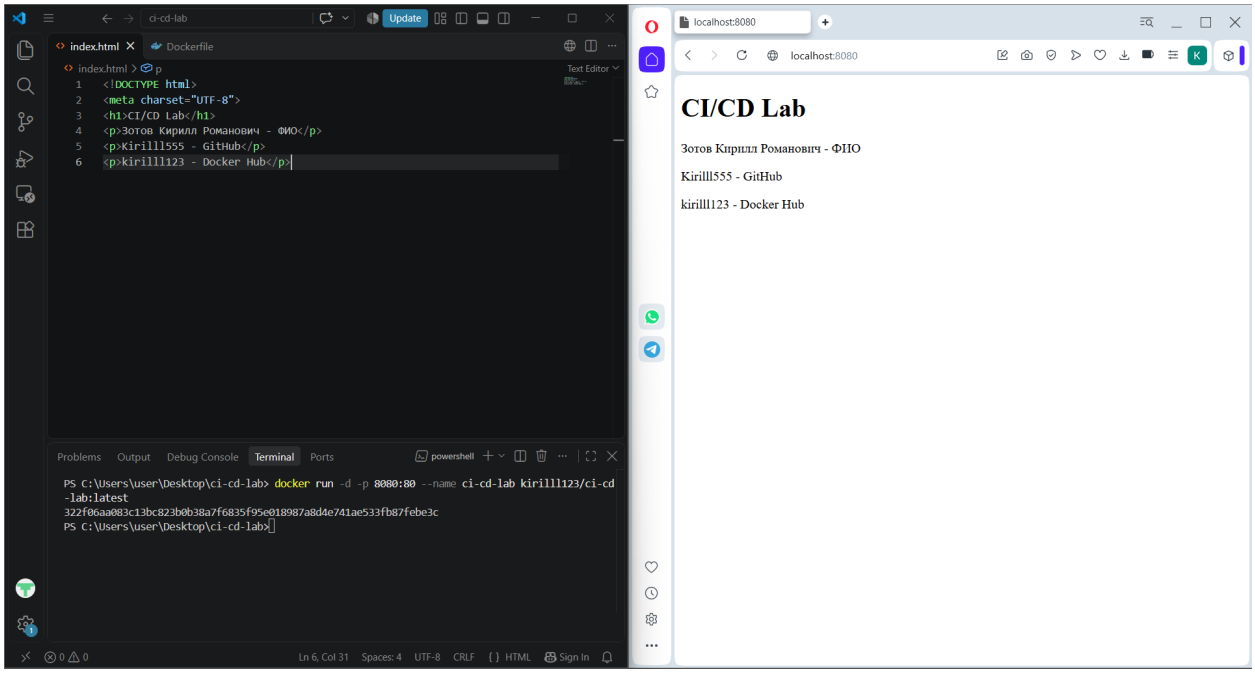
Команда:

docker build -t kirilll123/ci-cd-lab:latest .

Комментарий:

Собрал Docker-образ для своего сайта. В команде указал имя образа (kirilll123/ci-cd-lab) и точку в конце, чтобы Docker искал Dockerfile в текущей папке. Сборка завершилась успешно.

1.2



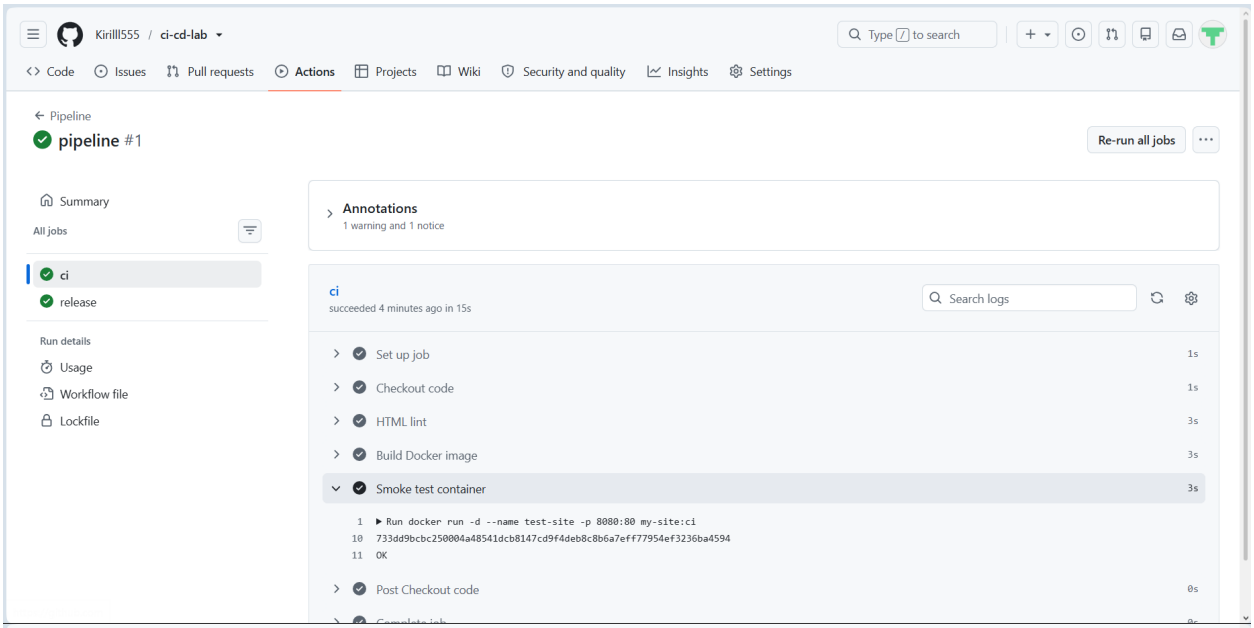
Команда:

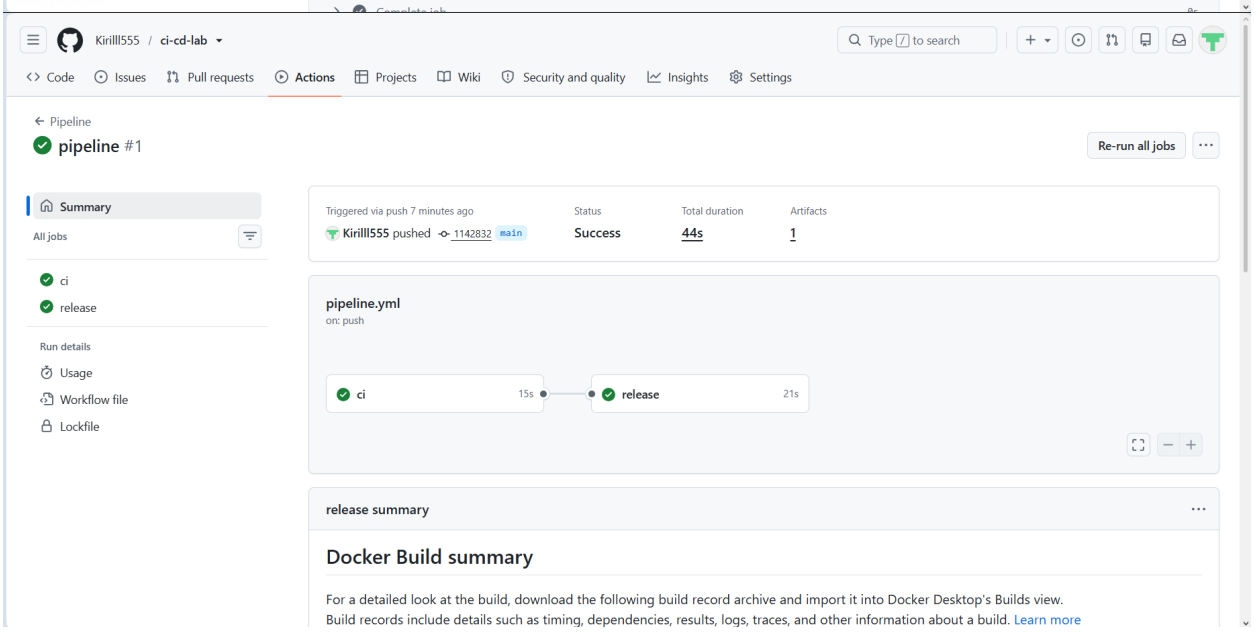
docker run -d -p 8080:80 --name ci-cd-lab kirilll123/ci-cd-lab:latest

Комментарий:

Запустил собранный контейнер в фоновом режиме. Пробросил порт 8080 с моего компьютера на 80-й порт внутри контейнера. Открыл в браузере localhost:8080 — сайт успешно загрузился, видны ФИО и никнеймы.

1.3





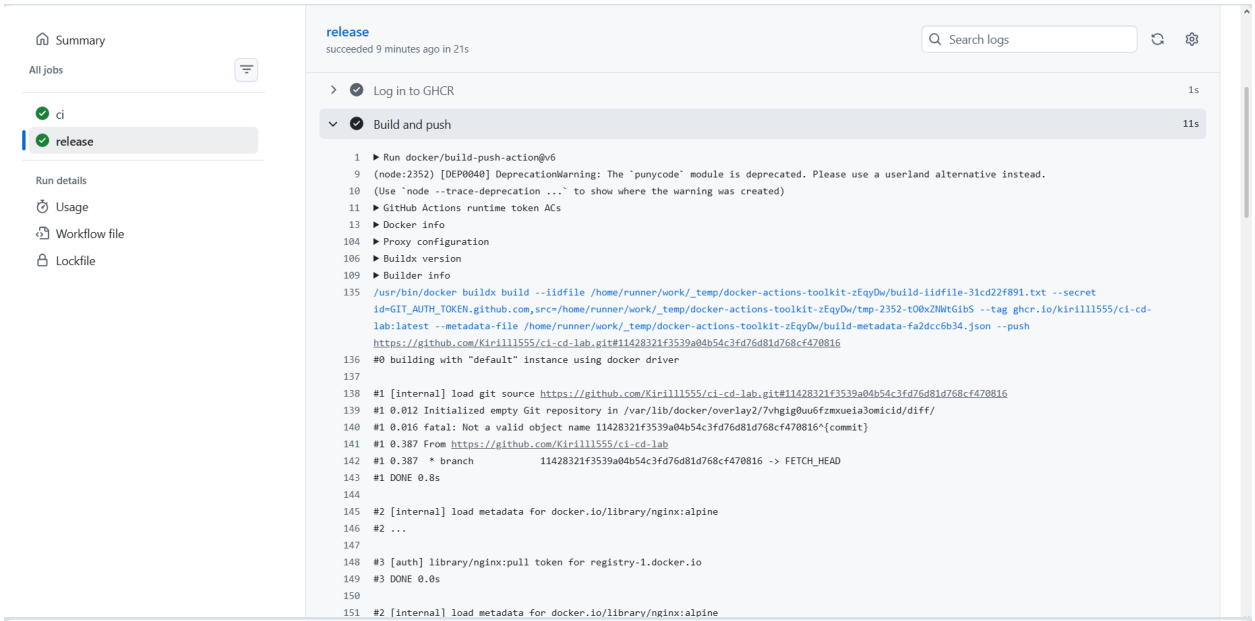
Действие:

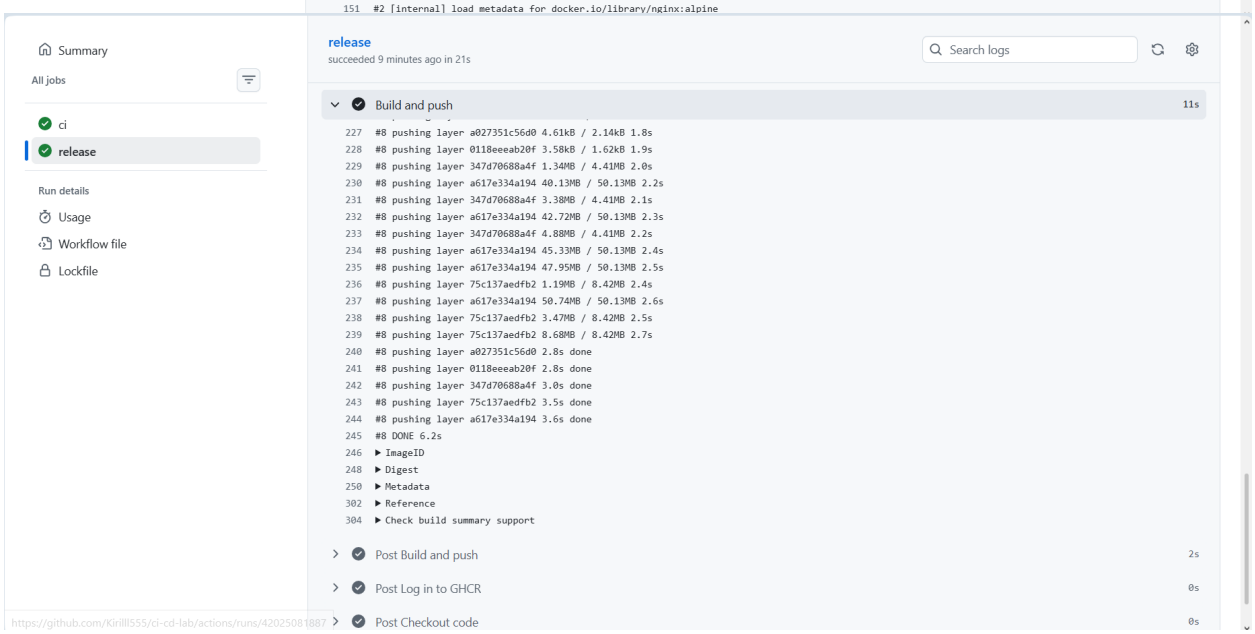
Сделать git push и проверить работу пайплайна на GitHub.

Комментарий:

После отправки кода на GitHub автоматически запустился workflow Pipeline. Джоб ci прошел успешно. В логах шага Smoke test container видно, что контейнер запустился, тест прошел и вывелась строка OK. В шапке запуска отображается мой ник и хеш коммита.

1.4





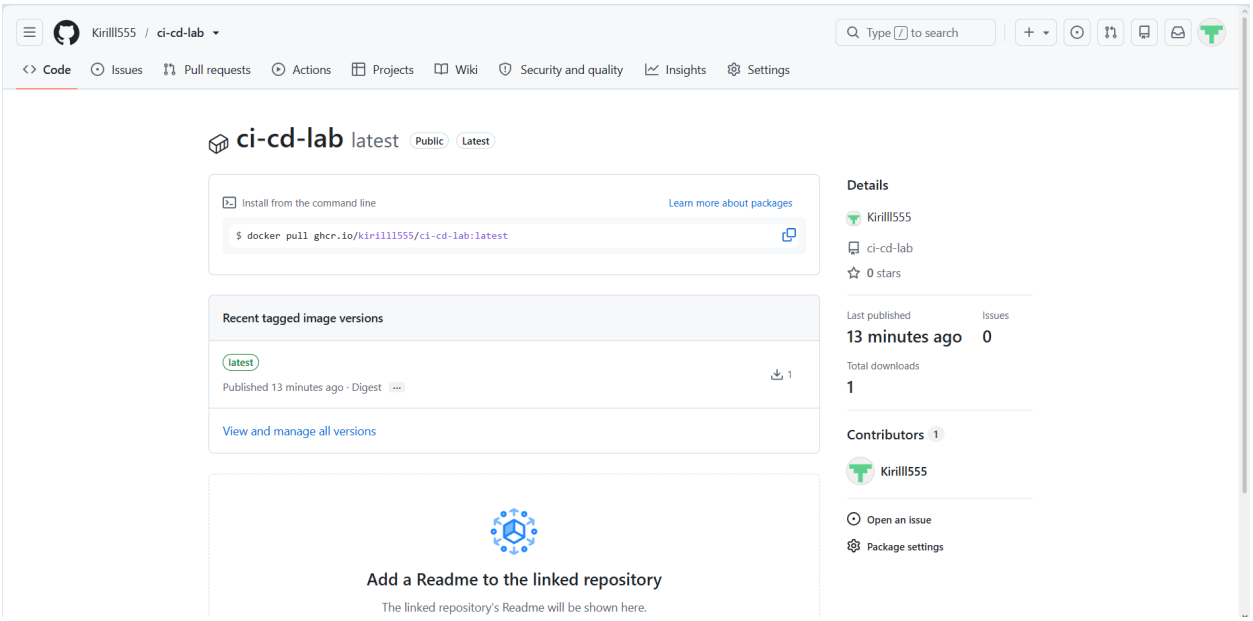
Действие:

Проверить джоб release в том же запуске Pipeline.

Комментарий:

Проверил работу джоба release. В логах шага Build and push видно, как Docker собирает образ и отправляет его в реестр GitHub Container Registry. Процесс завершился успешно, образ загружен.

1.5



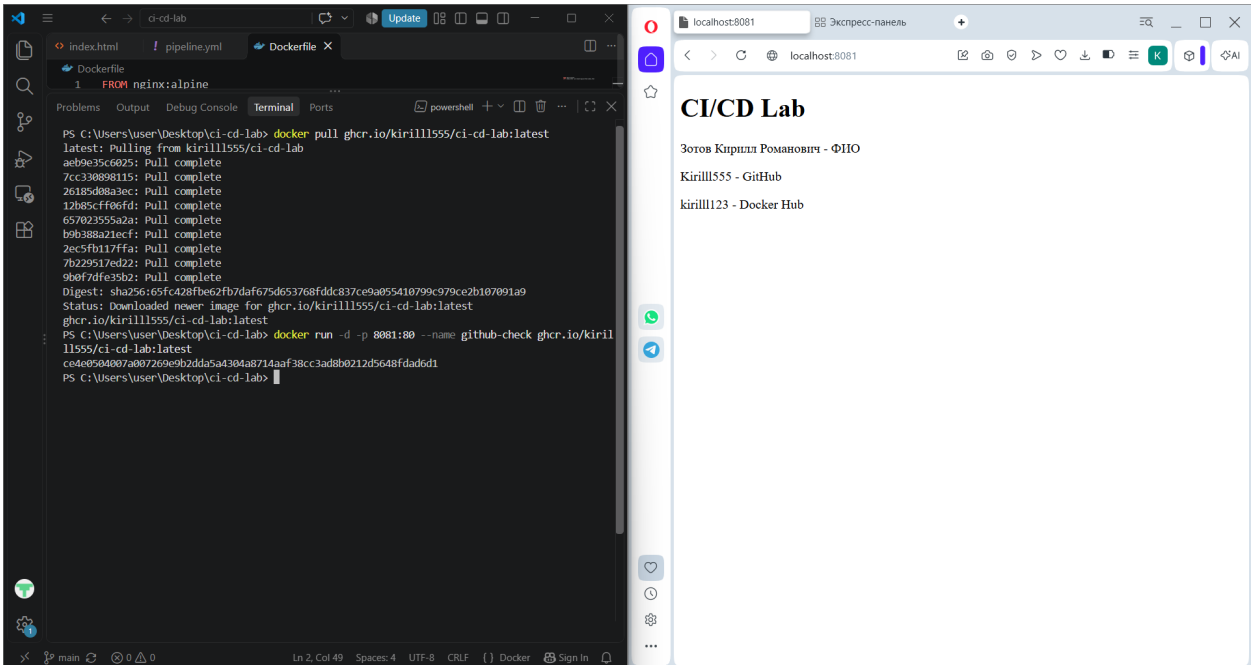
Действие:

Перейти на страницу Packages в репозитории GitHub.

Комментарий:

Перешел во вкладку Packages на GitHub. Убедился, что образ ci-cd-lab с тегом latest успешно опубликован. В пути пакета отображается мой никнейм.

1.6



Команда:

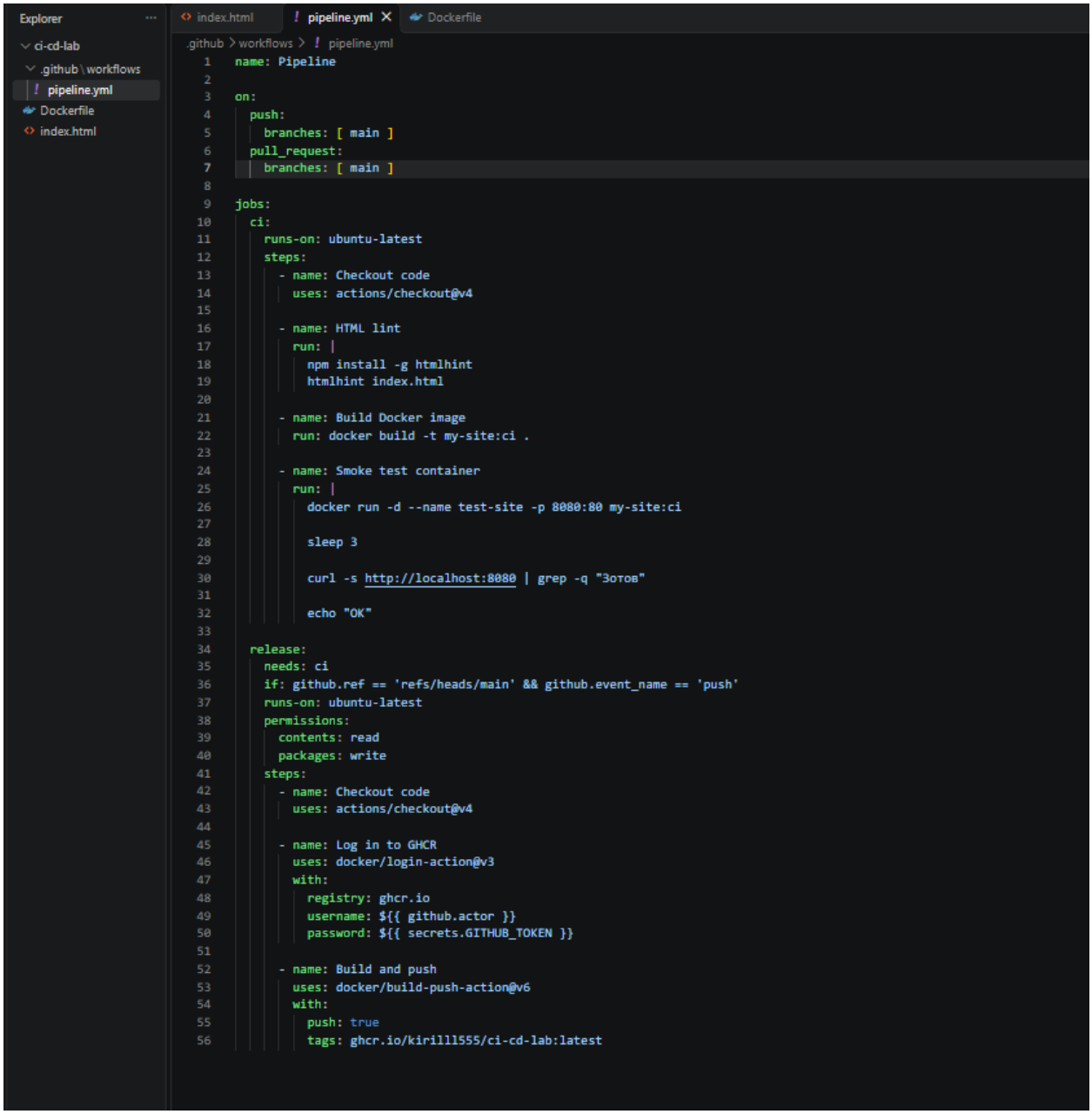
docker pull ghcr.io/kirilll555/ci-cd-lab:latest

docker run -d -p 8081:80 --name github-check ghcr.io/kirilll555/ci-cd-lab:latest

Комментарий:

Скачал образ обратно из GitHub с помощью docker pull. Затем запустил его в новом контейнере на 8081. Проверил в браузере — сайт успешно открывается, значит образ залился и скачался без ошибок.

2.1



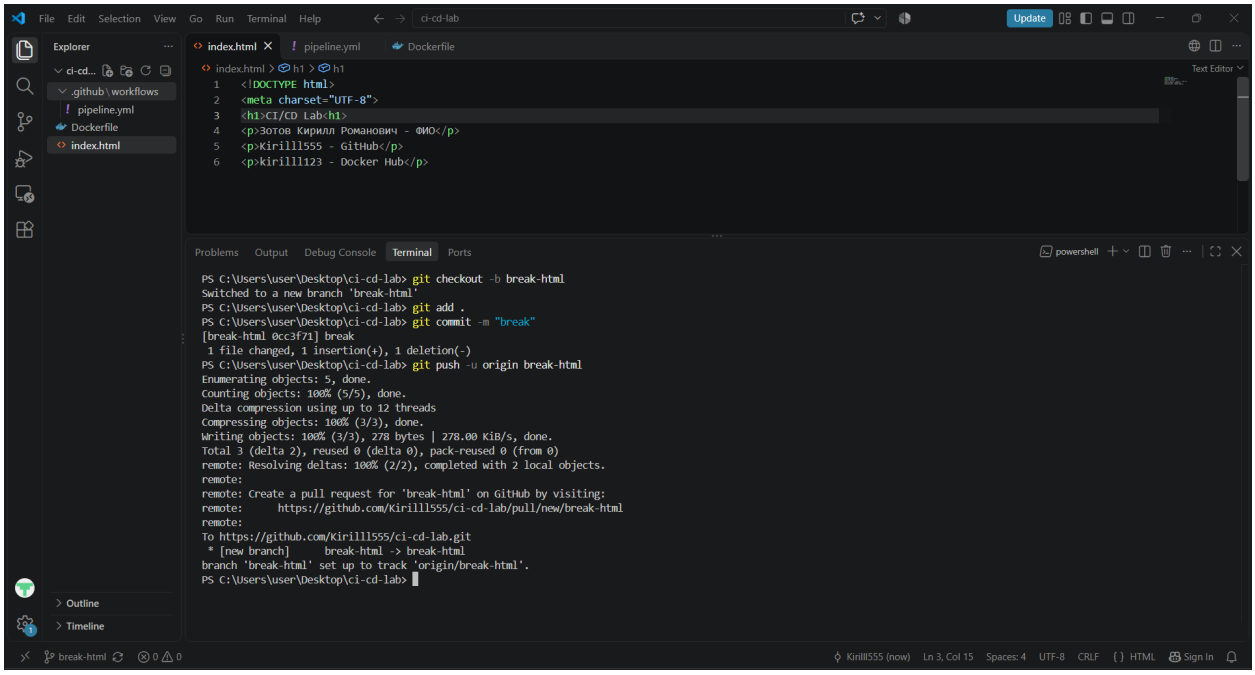
Действие:

Открыть файл pipeline.yml.

Комментарий:

Настроил пайплайн в файле pipeline.yml. Описал два джоба: ci и release. С помощью параметра needs: ci настроил зависимость, чтобы релиз запускался только после успешного прохождения тестов.

2.2



Команда:

git checkout -b break-html

git add .

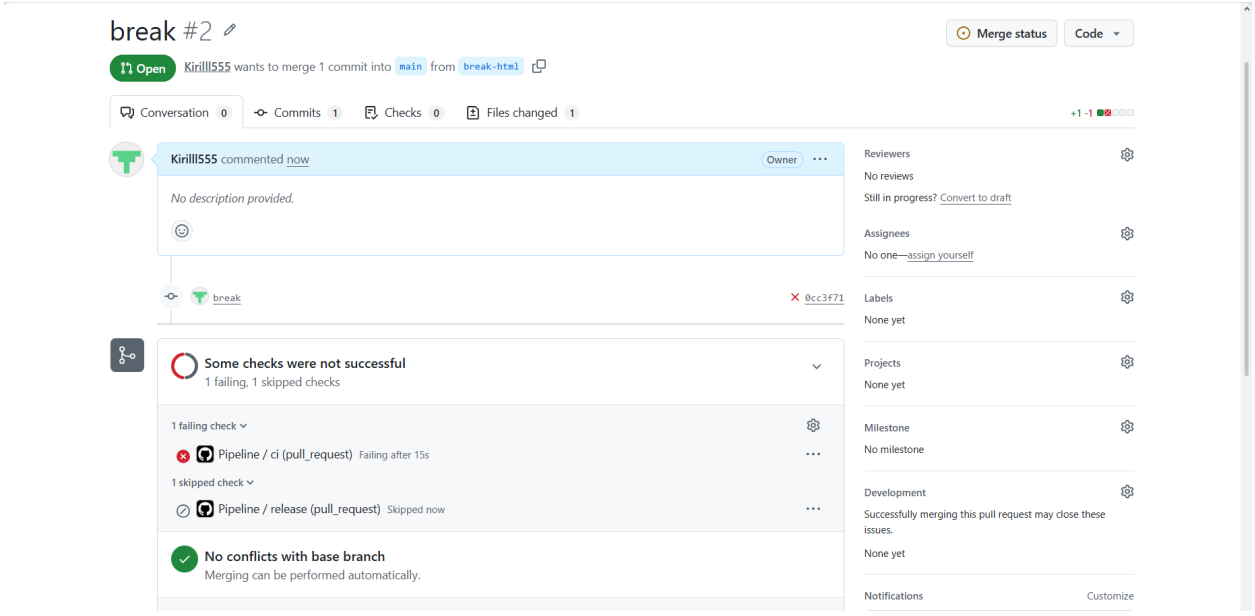
git commit -m "break "

git push -u origin break-html

Комментарий:

Создал отдельную ветку break-html для имитации ошибки. Специально сломал файл index.html (убрал закрывающий слэш в теге заголовка). Закоммитил это изменение и запушил ветку на GitHub, чтобы проверить, заблокирует ли CI слияние такого кода.

2.3 и 2.4



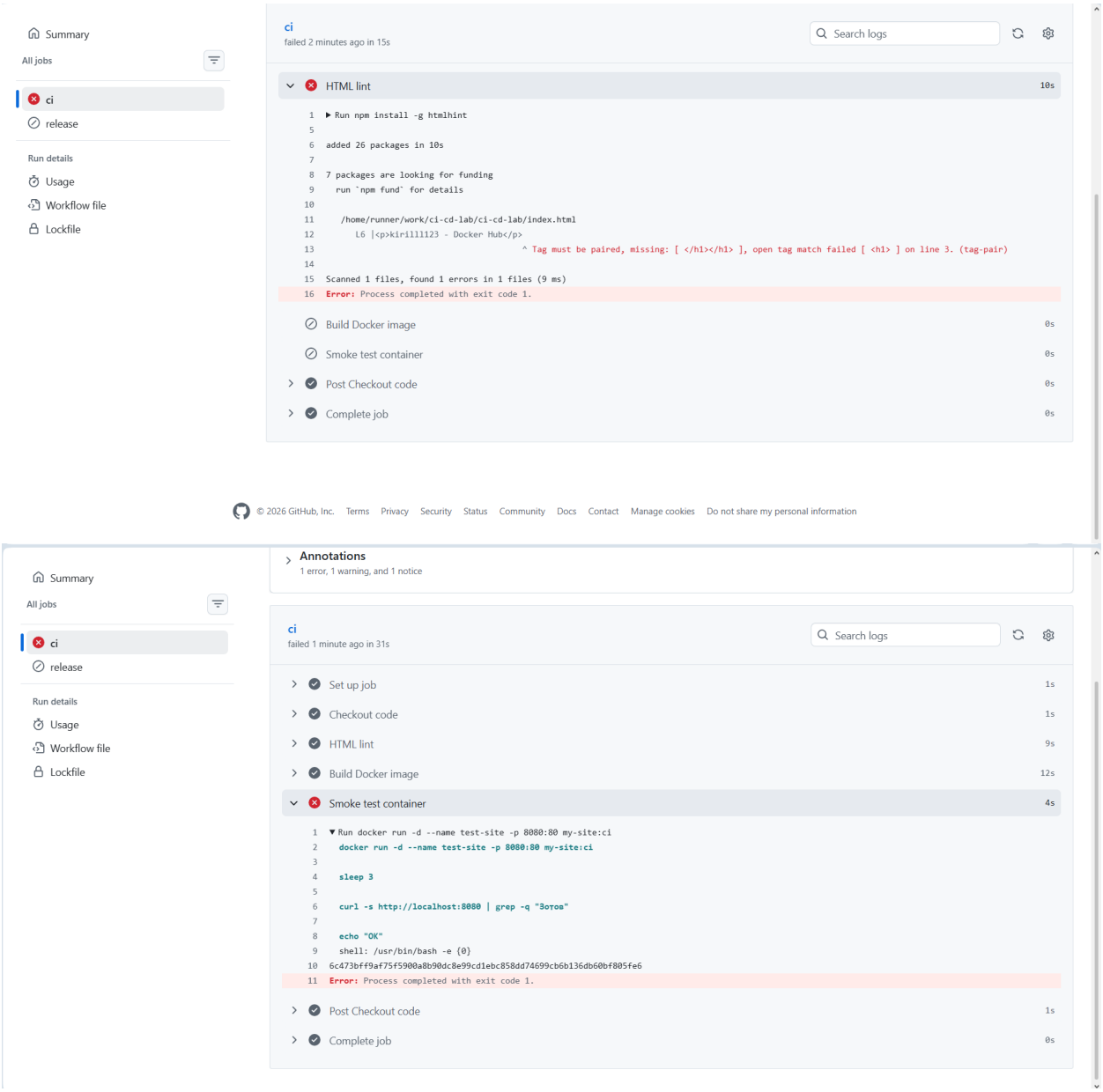
Действие:

Создать Pull Request из ветки break-html в main и проверить статус проверок.

Комментарий:

Создал Pull Request из ветки break-html в main. Так как в HTML-коде была допущена ошибка, автоматическая проверка ci не прошла и загорелась красным крестиком. При этом джоб release даже не начал выполняться и получил статус Skipped из-за зависимости. Это доказывает, что пайплайн блокирует слияние неверного кода и не позволяет собрать релизный образ, пока тесты не пройдут.

2.5



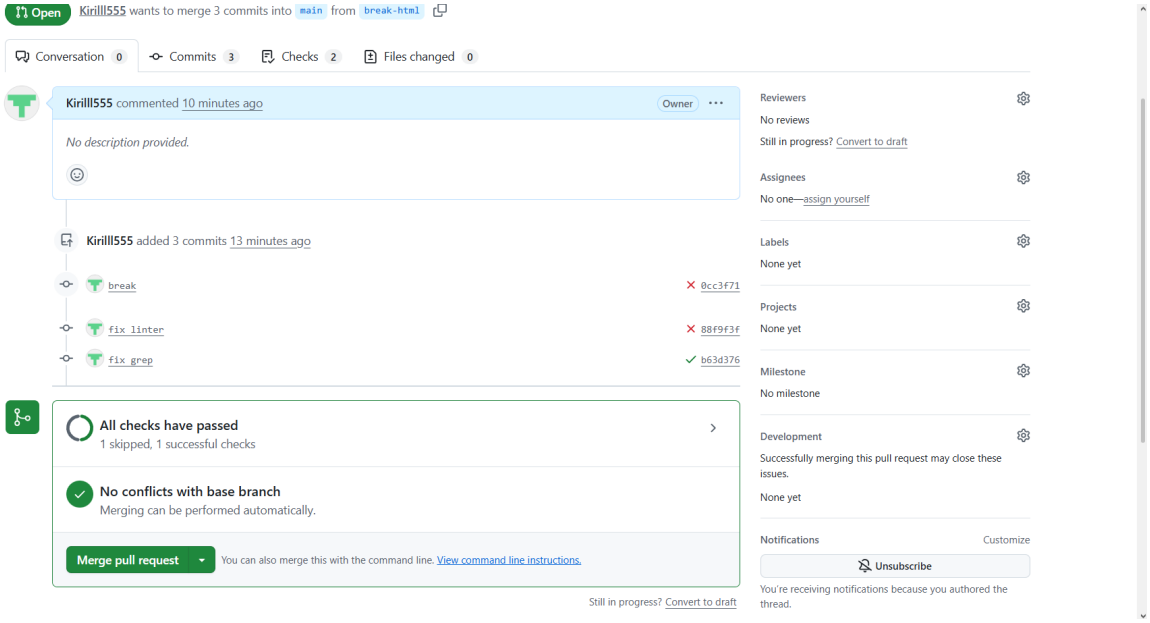
Действие:

Открыть логи упавшего джоба ci в Actions.

Комментарий:

Зашел в логи упавшей проверки ci, чтобы посмотреть причину. На первом скриншоте видно, как тест упал на шаге HTML lint — линтер htmlhint обнаружил незакрытый тег и вернул код ошибки 1. На втором скриншоте я специально исправил тег, но убрал из текста свою фамилию, чтобы проверить сам Smoke test. Видно, что команда не нашла совпадений и завершилась с ошибкой Process completed with exit code 1.

2.6



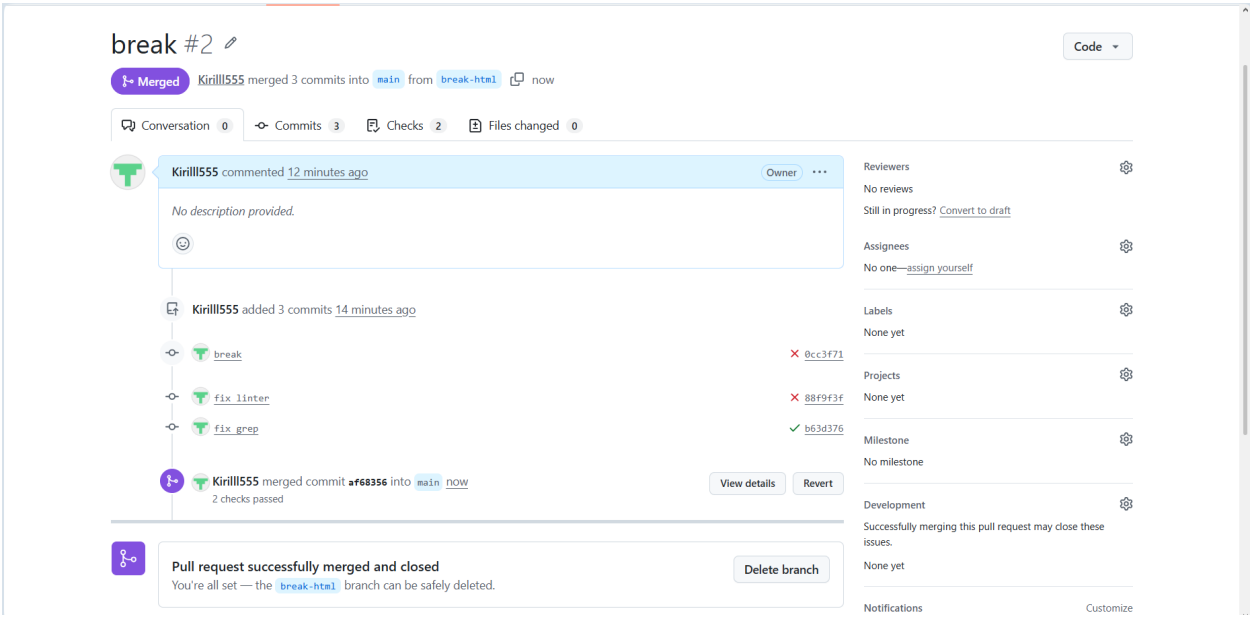
Действие:

Вернуть корректный HTML, запушить изменения и дождаться успешной проверки.

Комментарий:

Исправил код: вернул фамилию в текст и оставил полностью корректный файл. После пуша автоматическая проверка CI запустилась заново. На скриншоте видно, что все проверки прошли успешно, загорелись зеленые галочки и появилась надпись "All checks have passed".

2.7



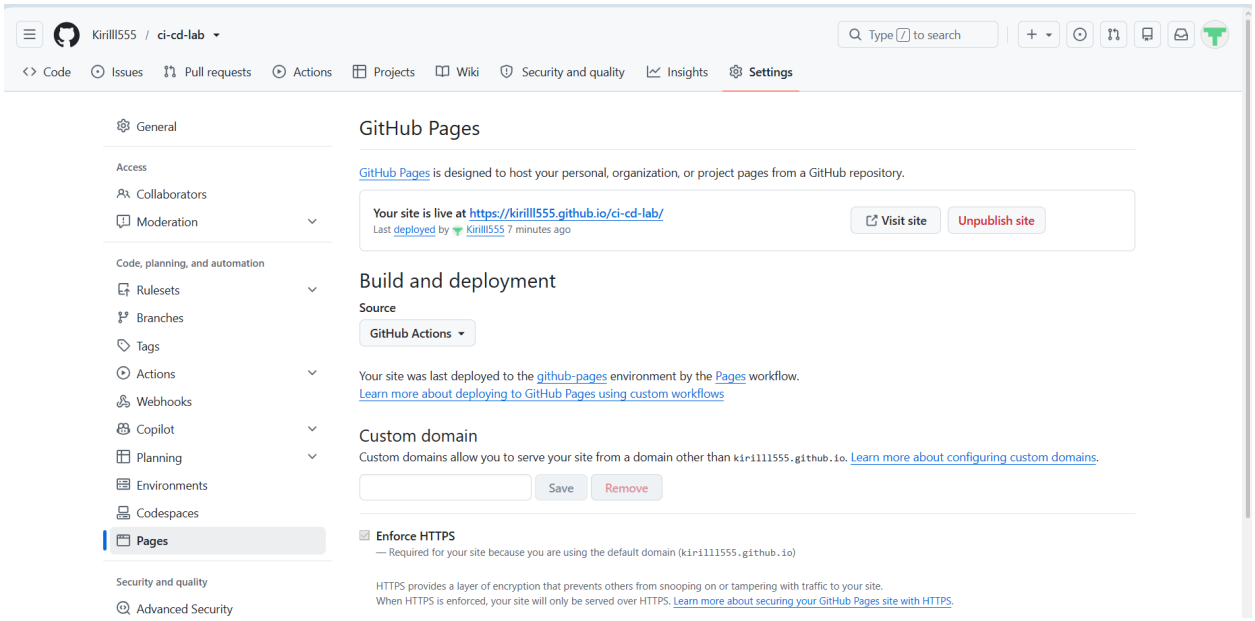
Действие:

Нажать кнопку Merge pull request и подтвердить слияние.

Комментарий:

Так как все проверки прошли успешно, я слил ветку break-html в основную ветку main. Появилась фиолетовая надпись «Merged», что означает успешное завершение слияния и готовность кода к деплою.

3.1



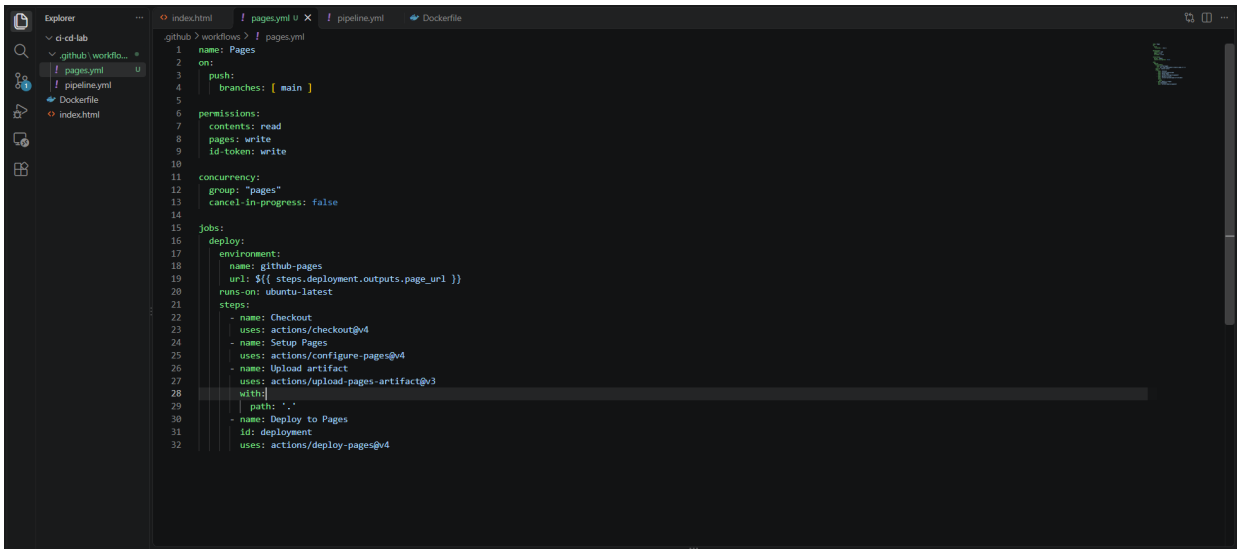
Действие:

Перейти в настройки репозитория и изменить источник сборки.

Комментарий:

В настройках репозитория переключил источник сборки с обычной ветки на GitHub Actions. Благодаря этому сайт теперь будет публиковаться автоматически через настроенный workflow. Сверху появилась плашка «Your site is live at...» с публичной ссылкой на сайт.

3.2



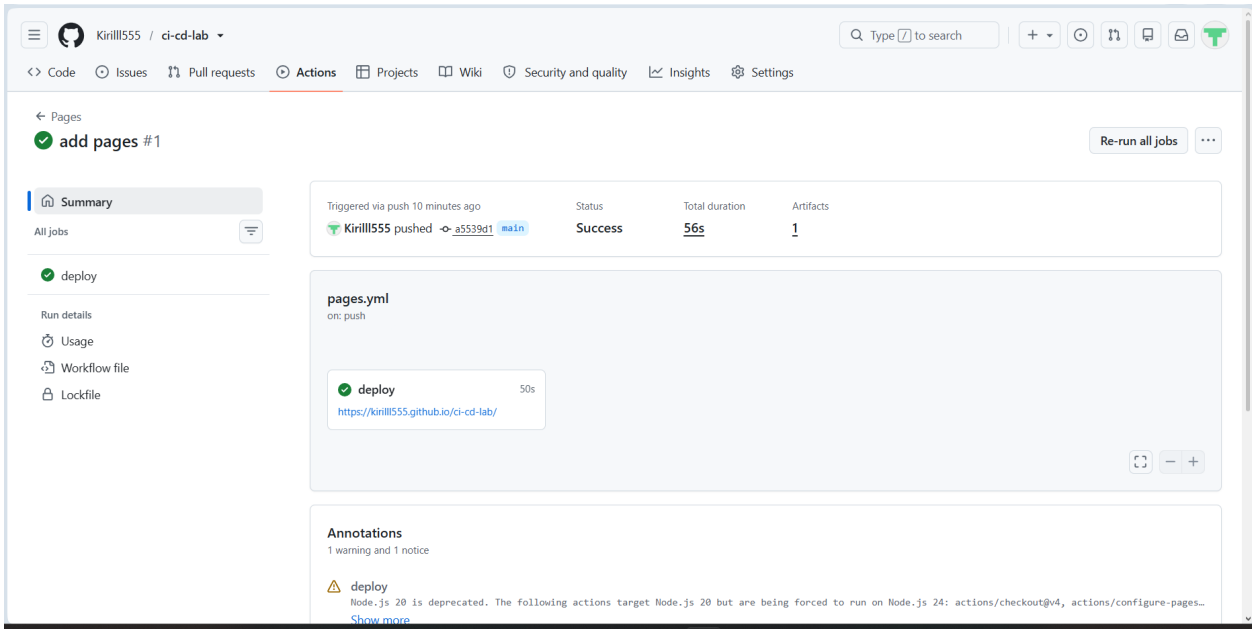
Действие:

Открыть файл pages.yml в VS Code.

Комментарий:

Создал отдельный workflow pages.yml для деплоя сайта. В нём прописал необходимые права доступа и шаги для того, чтобы робот забрал файлы из репозитория и опубликовал их на серверах GitHub Pages.

3.3



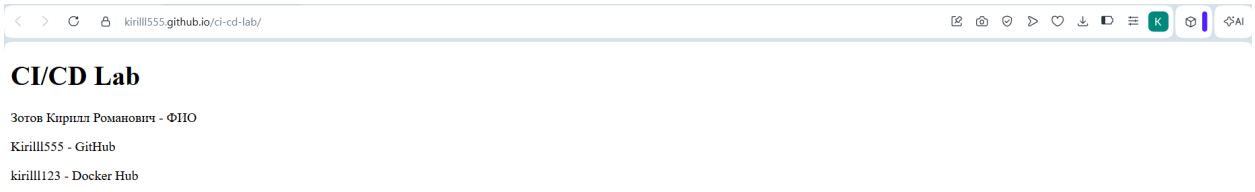
Действие:

Открыть вкладку Actions и проверить лог workflow Pages.

Комментарий:

Проверил работу workflow Pages. На странице Summary виден успешный запуск, а под ним — публичная ссылка на сайт. Это подтверждает, что файлы были успешно опубликованы на серверах GitHub.

3.4



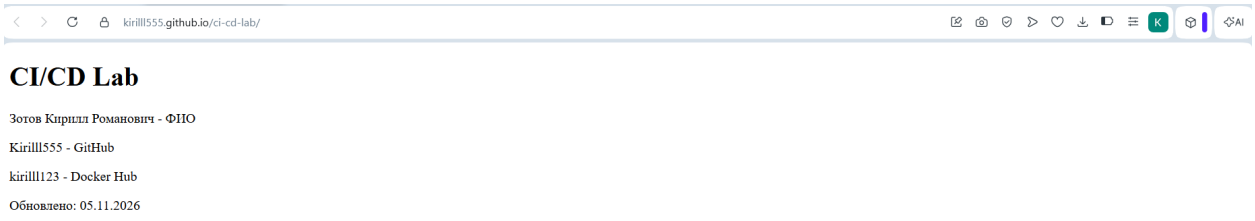
Действие:

Открыть публичную ссылку в браузере.

Комментарий:

Перешел по полученной ссылке. Сайт успешно открывается в браузере, адресная строка ведет на github.io. На странице отображаются все требуемые данные.

3.5



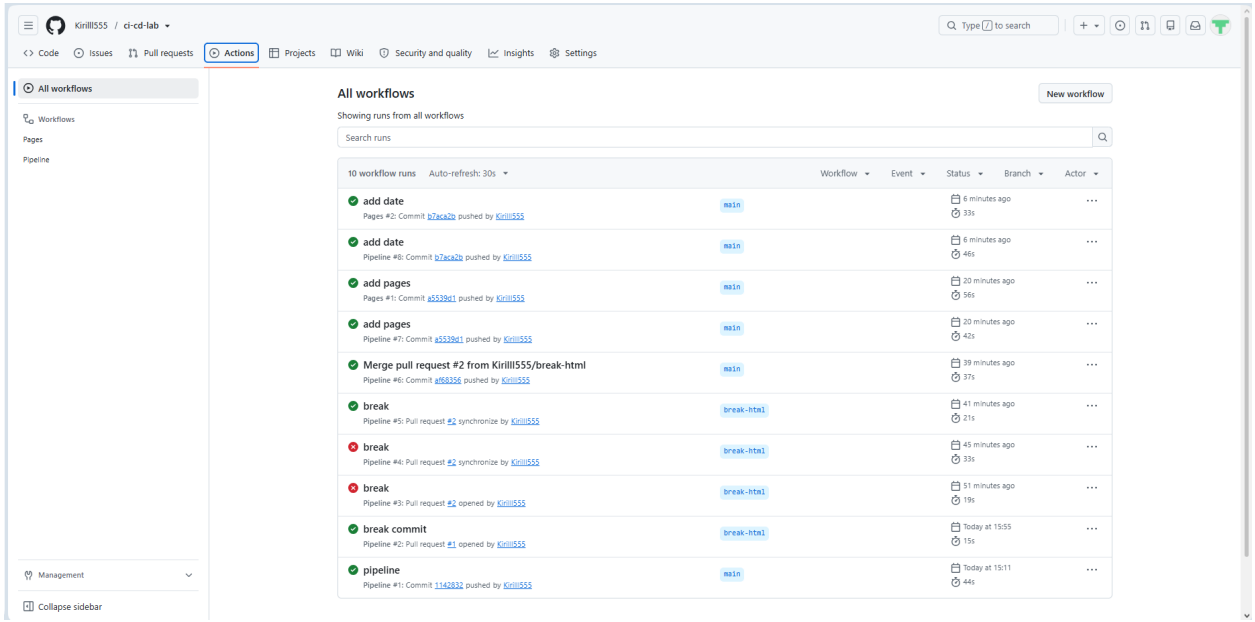
Действие:

Изменить index.html, запушить изменения и обновить страницу в браузере.

Комментарий:

Проверил автоматическое обновление. Внес изменение в файл index.html и запушил его в ветку main. Дождался, пока в Actions отработает новый деплой, и обновил страницу. Новая строчка "Обновлено: 05.11.2026" успешно появилась на сайте, что доказывает работу CI/CD.

4.1



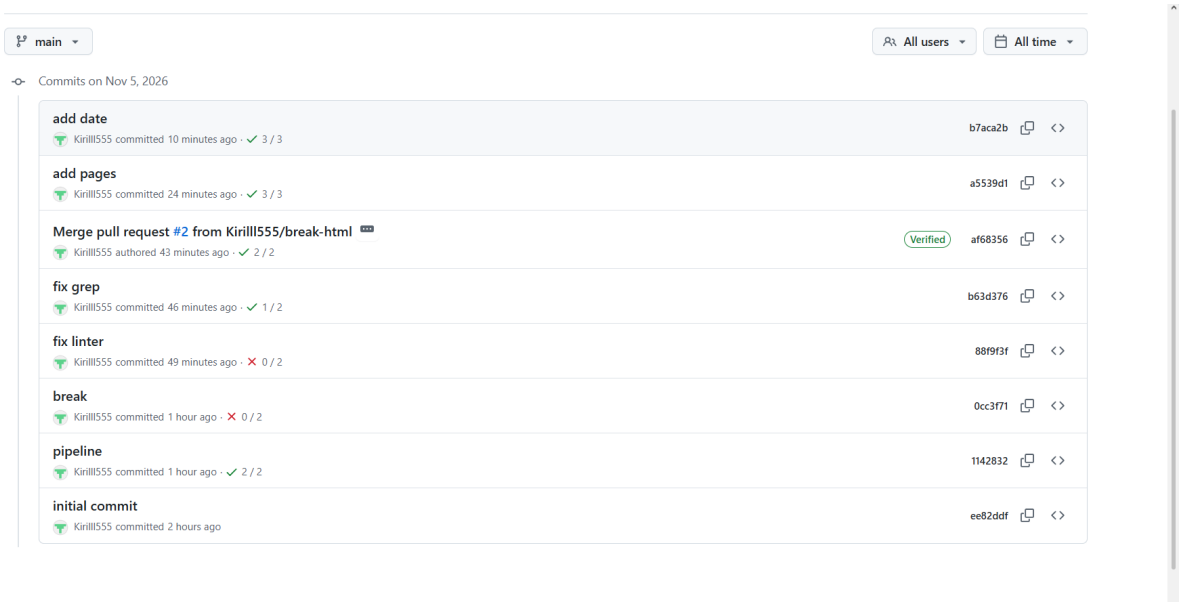
Действие:

Открыть вкладку Actions и посмотреть на список запусков на ветке main.

Комментарий:

Проверил вкладку Actions. На скриншоте видно, что после слияния Pull Request на ветке main подряд прошли все необходимые процессы: Pipeline и Pages. Все запуски отмечены зелеными галочками. Это доказывает, что полный цикл CI/CD работает автоматически.

4.2



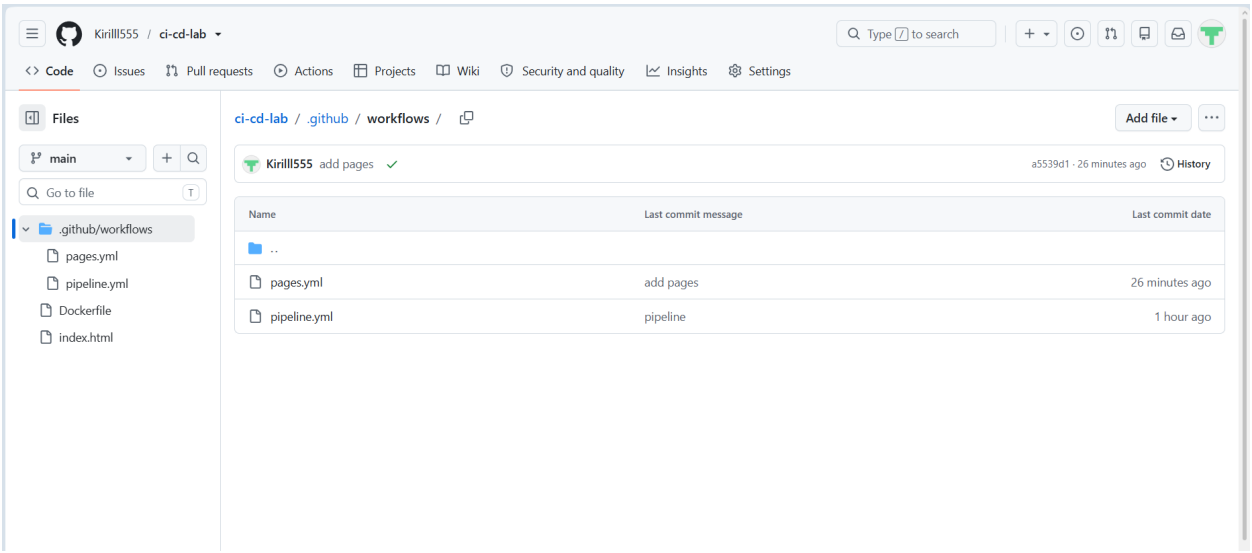
Действие:

Перейти на главную страницу репозитория и открыть историю коммитов.

Комментарий:

Открыл историю коммитов в репозитории. Здесь отражен весь процесс работы: от первоначального коммита до настройки CI/CD. Также видно, как я специально ломал код, а затем исправлял ошибки, после чего добавлял деплой и обновлял сайт.

4.3



Действие:

Открыть папку .github/workflows в репозитории на GitHub.

Комментарий:

Показал итоговую структуру файлов на GitHub. В корне проекта лежат файл index.html и Dockerfile. А в папке .github/workflows находятся оба конфигурационных файла, которые управляют автоматизацией: pipeline.yml и pages.yml.